

B.TECH – 1ST YEAR | COMPUTER SCIENCE & ENGINEERING

DATA STRUCTURES

UNIT I – LINEAR DATA STRUCTURES

Comprehensive Study Notes & Exam Guide

 Theory + Algorithms

 Diagrams & Flowcharts

 Q&A Practice

 Quick Revision

Covers: Introduction · ADT · Complexity Analysis · Searching · Sorting

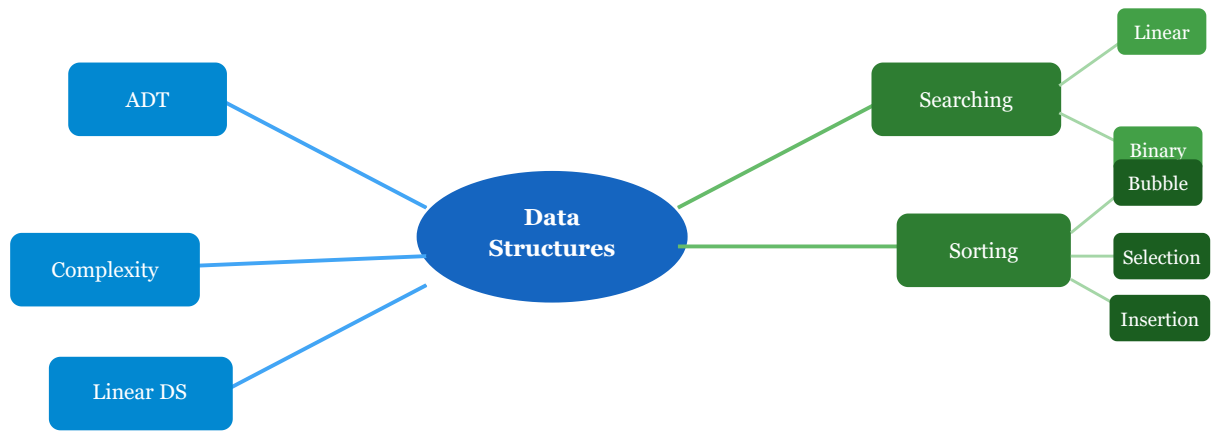
References: Weiss · Horowitz & Sahni · Cormen · Sedgewick

Table of Contents

1	Introduction to Data Structures	3
2	Linear Data Structures Overview	4
3	Abstract Data Types (ADT)	5
4	Complexity Analysis	6
5	Time Complexity	7
6	Space Complexity	8
7	Linear Search	9
8	Binary Search	10
9	Sorting Techniques Overview	11
10	Bubble Sort	12
11	Selection Sort	14
12	Insertion Sort	16
13	Questions & Answers	18
14	Quick Revision Sheet	21
—	Textbook References	22

How to Use This Module: Each section follows the sequence: Definition → Representation → Working Principle → Complexity → Advantages/Disadvantages → Applications → Diagram. Exam-focused Q&A and a one-page revision sheet are provided at the end.

Fig 0.1 – Unit I Mind Map



1.1 Definition

A **Data Structure** is a systematic way of organizing, storing, and managing data in a computer so that it can be accessed and modified efficiently. It defines both the storage layout and the set of operations that can be performed on the data.

Formally: *A data structure is a collection of data values, the relationships among them, and the functions or operations that can be applied to the data.*

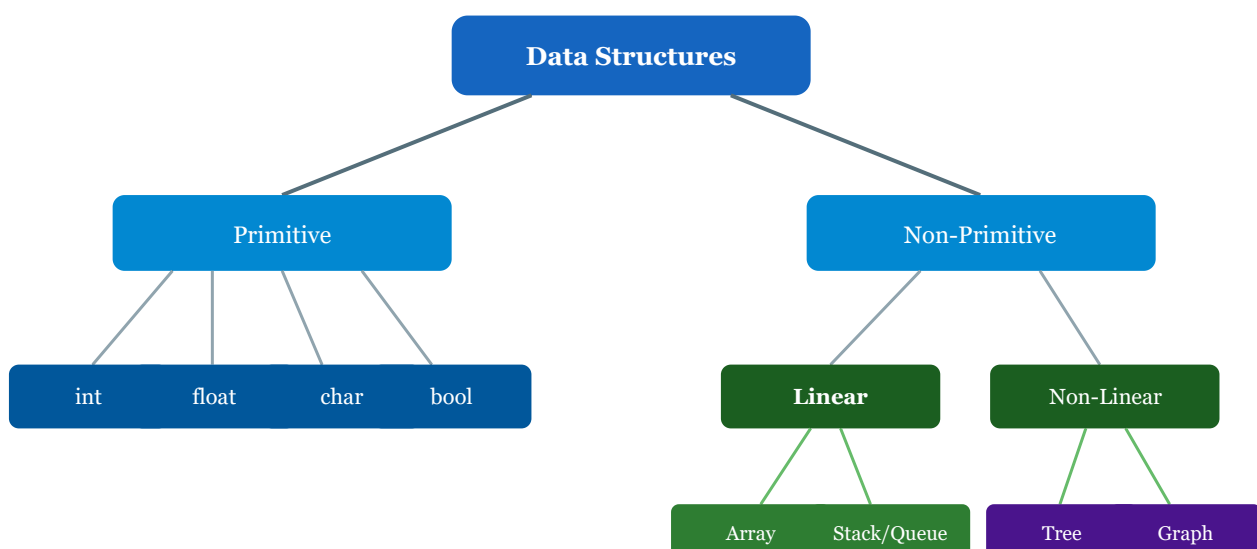
Key Insight: Choosing the right data structure directly determines the efficiency of an algorithm. An algorithm operating on a well-chosen data structure runs faster and uses less memory.

1.2 Importance of Data Structures

- Enables efficient data storage and retrieval.
- Reduces time and space complexity of algorithms.
- Fundamental to designing efficient software systems (databases, OS, compilers).
- Forms the backbone of competitive programming and technical interviews.
- Supports abstraction and modular programming.

1.3 Classification of Data Structures

Fig 1.1 – Classification of Data Structures



1.4 Operations on Data Structures

Operation	Description	Example
Traversal	Visit every element once	Print all array elements
Insertion	Add a new element	Push to stack
Deletion	Remove an element	Pop from stack
Searching	Find an element	Linear / Binary search
Sorting	Arrange in order	Bubble sort
Merging	Combine two structures	Merge two sorted arrays

2.1 Definition

A **Linear Data Structure** is one in which data elements are arranged *sequentially*, where each element is connected to its previous and next element. The elements form a single level of arrangement that can be traversed in a linear fashion.

2.2 Characteristics of Linear Data Structures

- **Sequential Organization:** Elements are stored one after another in memory.
- **Single Level:** All elements exist at the same level, unlike trees or graphs.
- **Easy Traversal:** Can be traversed in one pass (start to end or end to start).
- **Memory Allocation:** Can be static (arrays) or dynamic (linked lists).
- **Direct Access / Sequential Access:** Arrays support direct access; linked lists support sequential access.

2.3 Types of Linear Data Structures

Type	Storage	Access	Key Operations
Array	Contiguous	Random ($O(1)$)	Insert, Delete, Search
Linked List	Non-contiguous	Sequential ($O(n)$)	Insert, Delete, Traverse
Stack	Contiguous/Dynamic	LIFO	Push, Pop, Peek
Queue	Contiguous/Dynamic	FIFO	Enqueue, Dequeue

Fig 2.1 – Linear Data Structure Memory Representation

Array (Contiguous Memory)

Base Address → 1000

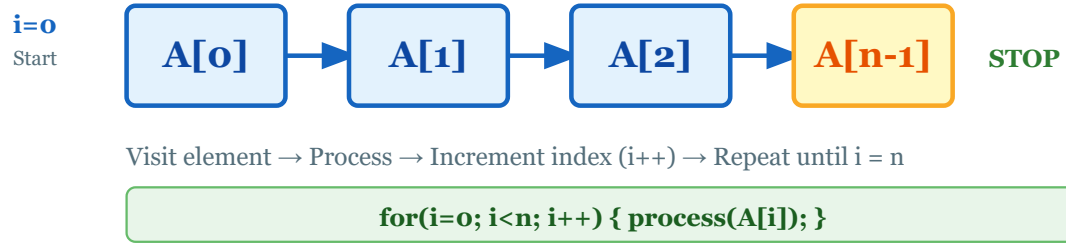


Linked List (Non-Contiguous Memory)



2.4 Array Traversal

Fig 2.2 – Array Traversal Diagram



§3 Abstract Data Types (ADT)

3.1 Concept of ADT

An **Abstract Data Type (ADT)** is a mathematical model for data types where the data type is defined by its behavior (semantics) from the user's point of view—specifically in terms of possible values, possible operations on data of this type, and the behavior of these operations—rather than by the implementation of these operations.

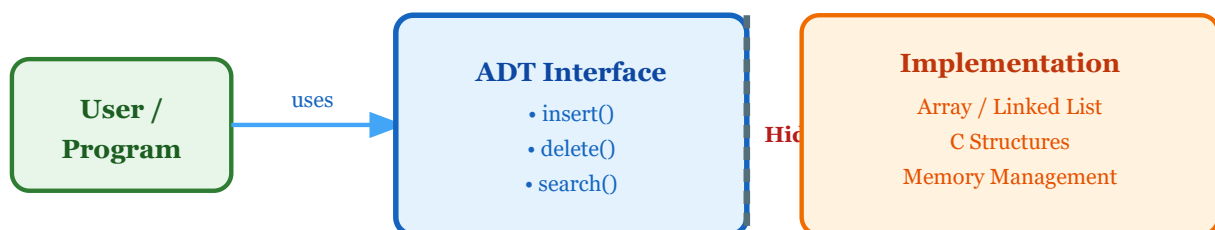
ADTs are "abstract" because they hide implementation details. The user only sees the interface (what operations are available), not how they are implemented.

Key Analogy: An ADT is like a TV remote. You press a button (operation) and get a result (behavior), without knowing the internal electronics (implementation).

3.2 Components of an ADT

- **Data:** The values or objects stored.
- **Operations:** Functions that can be performed on the data.
- **Preconditions:** Conditions that must hold before an operation executes.
- **Postconditions:** Conditions that hold after an operation executes.

Fig 3.1 – ADT Concept: Interface vs Implementation



3.3 Example ADT – Stack

A Stack ADT stores elements with LIFO (Last In, First Out) ordering. It defines:

- **Data:** Collection of elements + top pointer.
- **push(x):** Insert x at top. Precondition: stack not full.
- **pop():** Remove top element. Precondition: stack not empty.

- **peek()**: View top element without removing.
- **isEmpty()**: Returns true if stack is empty.

3.4 Implementation Using C

```
/* — Stack ADT Implementation in C — */
#include <stdio.h>
#define MAX 100

typedef struct {
    int data[MAX]; /* array to store elements */
    int top;       /* index of top element */
} Stack;

/* Initialize stack */
void init(Stack *s) {
    s->top = -1;
}

/* Check if stack is empty */
int isEmpty(Stack *s) {
    return s->top == -1;
}

/* Check if stack is full */
int isFull(Stack *s) {
    return s->top == MAX - 1;
}

/* Push operation */
void push(Stack *s, int x) {
    if (isFull(s)) { printf("Stack Overflow!\n"); return; }
    s->data[++(s->top)] = x;
    printf("Pushed: %d\n", x);
}

/* Pop operation */
int pop(Stack *s) {
    if (isEmpty(s)) { printf("Stack Underflow!\n"); return -1; }
    return s->data[(s->top)--];
}

/* Main */
int main() {
    Stack st;
    init(&st);
    push(&st, 10);
    push(&st, 20);
```

```
push(&st, 30);  
printf("Popped: %d\n", pop(&st));  
return 0;  
}
```

3.5 Advantages of ADT

- Promotes modularity and code reuse.
- Implementation can be changed without affecting users.
- Reduces complexity by hiding internal details.
- Easier debugging and maintenance.

4.1 What is Complexity Analysis?

Complexity Analysis is the process of determining how the time and space requirements of an algorithm scale with respect to the input size n . It allows us to compare algorithms independent of hardware, language, or implementation.

Complexity Notations

Big-O $O(f(n)) \rightarrow$ Upper bound (Worst case)

Omega $\Omega(f(n)) \rightarrow$ Lower bound (Best case)

Theta $\Theta(f(n)) \rightarrow$ Tight bound (Average case)

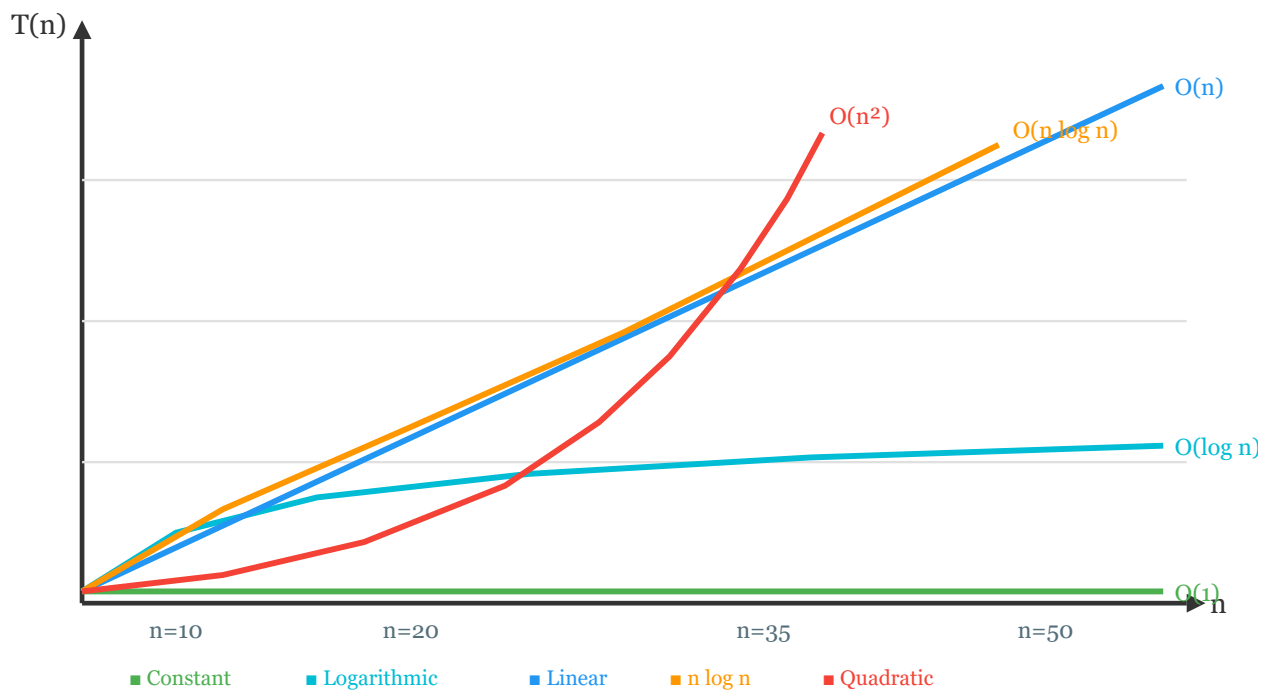
4.2 Big-O Notation

Big-O notation describes the upper bound of an algorithm's running time. It tells us the **worst-case scenario**—the maximum time (or space) the algorithm will ever take.

Formal Definition: $T(n) = O(f(n))$ if there exist constants $c > 0$ and n_0 such that $T(n) \leq c \cdot f(n)$ for all $n \geq n_0$.

Complexity	Name	Example	$n=100$ (ops)
$O(1)$	Constant	Array access	1
$O(\log n)$	Logarithmic	Binary Search	~ 7
$O(n)$	Linear	Linear Search	100
$O(n \log n)$	Linearithmic	Merge Sort	~ 700
$O(n^2)$	Quadratic	Bubble Sort	10,000
$O(2^n)$	Exponential	Tower of Hanoi	$\sim 1.27 \times 10^{30}$

Fig 4.1 – Time Complexity Growth Rate Graph



§5 Time Complexity

5.1 Definition

Time Complexity of an algorithm is the number of basic operations (comparisons, assignments) performed as a function of the input size n . It measures how quickly the algorithm's execution time grows as the input grows.

Note: Time complexity does NOT measure actual clock time—it measures the *number of operations*, which is hardware-independent.

5.2 Counting Operations – Example

```
/* Count steps for this code: */
int sum = 0;                /* 1 step */
for(int i = 0; i < n; i++) { /* n+1 steps (loop condition) */
    sum = sum + i;          /* n steps */
}
printf("%d", sum);          /* 1 step */

/* Total = 1 + (n+1) + n + 1 = 2n + 3 → O(n) */
```

5.3 Rules for Calculating Time Complexity

- **Simple statement:** $O(1)$ – constant time.
- **Sequential statements:** Add complexities $\rightarrow O(f) + O(g) = O(\max(f, g))$.
- **Single loop (1 to n):** $O(n)$.
- **Nested loops (both 1 to n):** $O(n^2)$.
- **Loop where i doubles each time:** $O(\log n)$.

Time Complexity Examples

```
x = x + 1; → O(1)
for(i=0; i<n; i++) x++; → O(n)
for(i=0; i<n; i++) → O(n^2)
    for(j=0; j<n; j++) x++;
for(i=1; i<n; i*=2) x++; → O(log n)
for(i=0; i<n; i++) → O(n log n)
```

```
for (j=1; j<n; j*=2) x++;
```

5.4 Best, Worst, and Average Cases

Case	Description	Notation	Example (Linear Search)
Best Case	Minimum operations needed	$\Omega(1)$	Element is at index 0
Average Case	Expected operations	$\Theta(n/2)$	Element is in middle
Worst Case	Maximum operations needed	$O(n)$	Element not found or at end

5.5 Advantages of Analyzing Time Complexity

- Predicts algorithm performance before running.
- Helps compare algorithms independently of hardware.
- Identifies bottlenecks in code.
- Guides selection of optimal algorithms for large inputs.

5.6 Disadvantages

- Does not account for actual execution time (constants and lower-order terms are ignored).
- Asymptotic analysis may be misleading for small inputs.

6.1 Definition

Space Complexity is the amount of memory an algorithm uses as a function of input size n . It includes:

- **Fixed Part (c):** Memory for constants, simple variables, fixed-size arrays, program code. Independent of input.
- **Variable Part $S(n)$:** Memory that grows with input size—depends on n .

📌 Space Complexity Formula

Total Space = Fixed Part + Variable Part

$$S(P) = c + S_p(n)$$

where c is constant memory and $S_p(n)$ is variable memory

6.2 Examples

Algorithm	Space Complexity	Reason
Swap two numbers	$O(1)$	Only 3 variables (a, b, temp)
Sum of n elements	$O(n)$	Array of n elements stored
Linear Search	$O(1)$	No extra array; only loop variable
Binary Search (iterative)	$O(1)$	Only low, high, mid variables
Binary Search (recursive)	$O(\log n)$	Recursive call stack
Bubble Sort	$O(1)$	In-place, only temp swap variable
Merge Sort	$O(n)$	Extra array for merging

6.3 Space-Time Trade-off

In algorithm design, there is often a trade-off between time and space:

- Using more memory can make an algorithm faster.
- Reducing memory usage may require more computation time.

- Example: Memoization uses extra space ($O(n)$) to reduce time from $O(2^n)$ to $O(n)$.

Trade-off Example: Fibonacci – Recursive approach: $O(2^n)$ time, $O(n)$ space.

Dynamic Programming: $O(n)$ time, $O(n)$ space. Iterative with two variables: $O(n)$ time, $O(1)$ space.

6.4 Applications

- Memory-constrained embedded systems (prefer low space algorithms).
- Database query optimization.
- Mobile applications with limited RAM.
- Real-time systems requiring predictable memory usage.

7.1 Definition

Linear Search (also called Sequential Search) is the simplest searching algorithm that checks every element of the list sequentially until the desired element is found or the list is exhausted.

7.2 Algorithm

```
/* Linear Search Algorithm */
Algorithm LinearSearch(A[], n, key):
    Input : Array A of n elements, search key
    Output: Index of key if found, else -1

    for i = 0 to n-1 do
        if A[i] == key then
            return i      /* Element found at index i */
        end if
    end for
    return -1             /* Element not found */
```

7.3 C Implementation

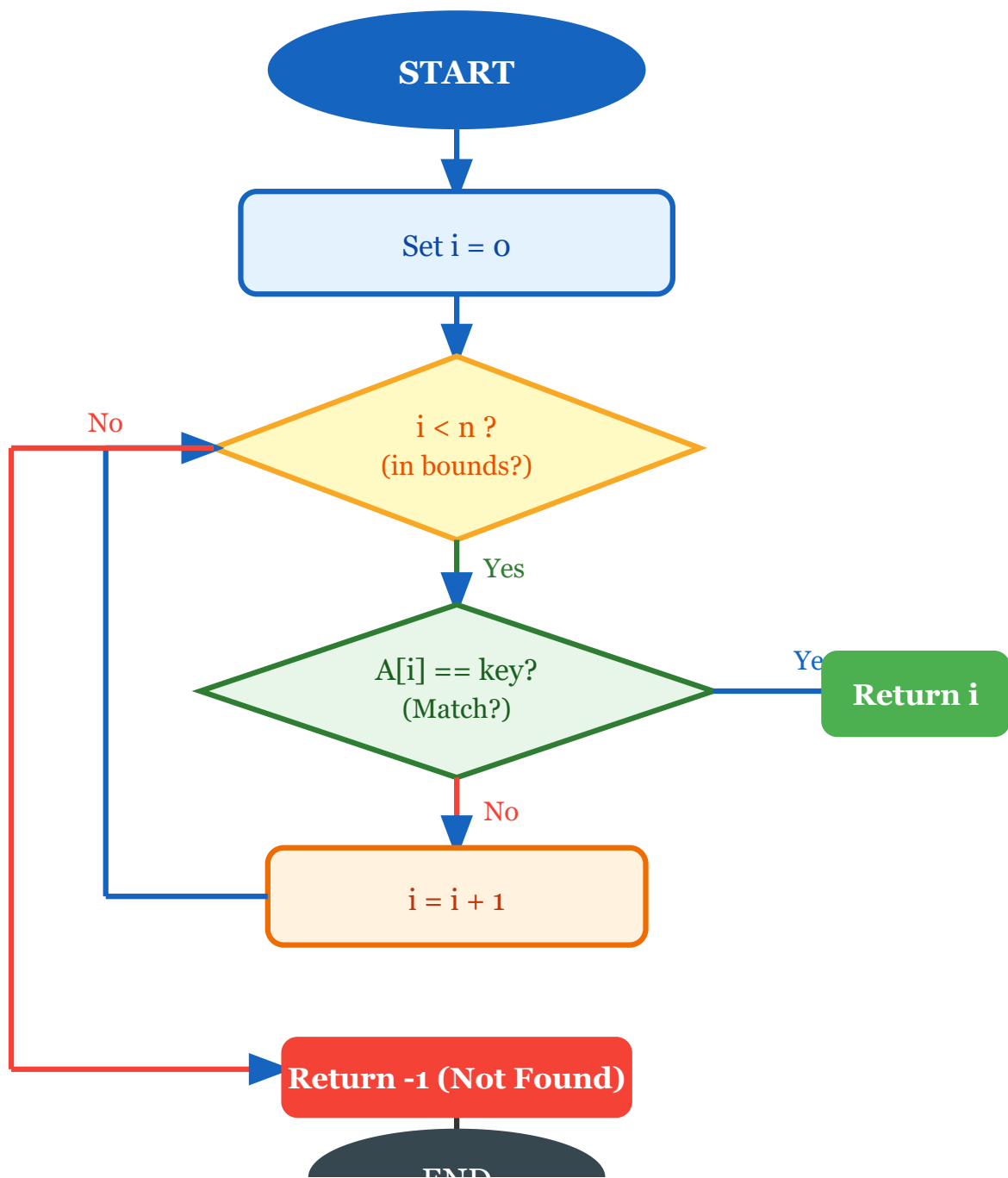
```
#include <stdio.h>

int linearSearch(int A[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (A[i] == key)
            return i;    /* found at index i */
    }
    return -1;          /* not found */
}

int main() {
    int A[] = {10, 25, 37, 49, 62, 78};
    int n = 6, key = 49;
    int result = linearSearch(A, n, key);
    if (result != -1)
        printf("Found at index %d\n", result);
    else
        printf("Not found\n");
}
```

```
return 0;  
}
```

Fig 7.1 – Linear Search Flowchart



Linear Search Complexity

Best Case : $O(1)$ - Element found at index 0

Average : $O(n/2)$ - Element in middle

Worst Case : $O(n)$ - Element at end or not found

Space : $O(1)$ - In-place (no extra memory)

7.4 Advantages & Disadvantages

Advantages	Disadvantages
Works on unsorted arrays	Very slow for large arrays: $O(n)$
Simple to implement	Not suitable for repeated searches
No preprocessing required	Inefficient for sorted data
Works on any data structure	Impractical for large datasets

7.5 Applications

- Searching in small or unsorted arrays.
- Finding an element in a linked list.
- Searching in files not organized sequentially.
- Used when elements are few (< 50).

§8 Binary Search

8.1 Definition

Binary Search is an efficient algorithm for finding an element in a *sorted* array. It uses a divide-and-conquer strategy, repeatedly halving the search space until the target is found or the search space is empty.

Prerequisite: The array **MUST** be sorted in ascending (or descending) order before applying binary search.

8.2 Algorithm

```
Algorithm BinarySearch(A[], low, high, key):
    while low <= high do
        mid = (low + high) / 2
        if A[mid] == key then
            return mid          /* Found! */
        else if A[mid] < key then
            low = mid + 1       /* Search right half */
        else
            high = mid - 1      /* Search left half */
        end if
    end while
    return -1                  /* Not found */
```

8.3 C Implementation

```
#include <stdio.h>

int binarySearch(int A[], int n, int key) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (A[mid] == key) return mid;
        else if (A[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int main() {
```

```

int A[] = {10, 20, 30, 40, 50, 60, 70};
int n = 7, key = 40;
int result = binarySearch(A, n, key);
if (result != -1) printf("Found at index: %d\n", result);
else
    printf("Not found\n");
return 0;
}

```

8.4 Step-by-Step Trace: Search key=40 in {10,20,30,40,50,60,70}

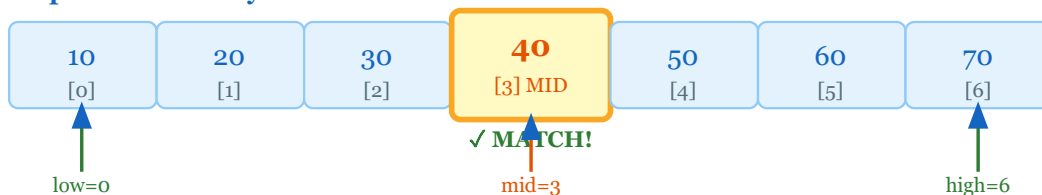
Step	low	high	mid	A[mid]	Decision
1	0	6	3	40	Match! Return 3

Search key=25 in {10,20,30,40,50,60,70}:

Step	low	high	mid	A[mid]	Decision
1	0	6	3	40	$25 < 40 \rightarrow \text{high} = \text{mid} - 1 = 2$
2	0	2	1	20	$25 > 20 \rightarrow \text{low} = \text{mid} + 1 = 2$
3	2	2	2	30	$25 < 30 \rightarrow \text{high} = \text{mid} - 1 = 1$
4	2	1	—	—	low > high → Not Found (-1)

Fig 8.1 – Binary Search Process on Array {10,20,30,40,50,60,70}, key=40

Step 1: Entire Array



$$\text{mid} = (\text{low} + \text{high}) / 2 = (0 + 6) / 2 = 3$$

$A[3] = 40 == \text{key} (40) \rightarrow \text{FOUND at index } 3 \checkmark$

Time Complexity: $O(\log_2 n)$

For $n=7$: max comparisons = $\lceil \log_2 7 \rceil = 3$ comparisons

Binary Search Complexity

Best Case : $O(1)$ - Element is the middle element

Average : $O(\log n)$ - Typical case

Worst Case : $O(\log n)$ - Element not found or at end

Space (Iterative) : $O(1)$

Space (Recursive) : $O(\log n)$ - due to call stack

Max Comparisons : $\lceil \log_2(n+1) \rceil$

8.5 Comparison: Linear vs Binary Search

Feature	Linear Search	Binary Search
Prerequisite	None (works on unsorted)	Array must be sorted
Time (Worst)	$O(n)$	$O(\log n)$
Time (Best)	$O(1)$	$O(1)$
Space	$O(1)$	$O(1)$ iterative
Implementation	Simple	Moderate
Data Structure	Any	Array (random access)
Suitable for	Small/unsorted data	Large sorted data

9.1 Definition

Sorting is the process of arranging elements of a list in a specific order—typically ascending or descending—based on a comparison criterion. Sorting is a fundamental operation in computer science, serving as a preprocessing step for many other algorithms.

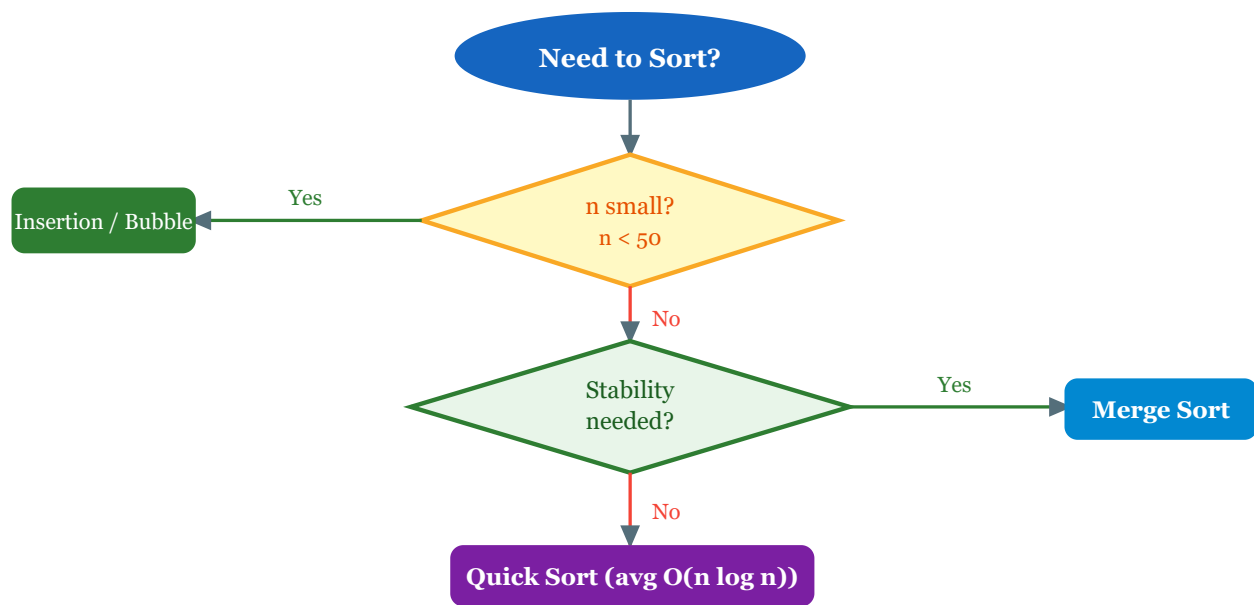
9.2 Classification of Sorting Algorithms

Property	Types
Based on memory	In-place ($O(1)$ space) vs Out-of-place (extra memory)
Based on stability	Stable (preserves equal element order) vs Unstable
Based on technique	Comparison-based vs Non-comparison-based
Based on recursion	Recursive (Merge, Quick) vs Iterative (Bubble, Selection)

9.3 Sorting Algorithm Comparison

Algorithm	Best Case	Avg Case	Worst Case	Space	Stable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No

Fig 9.1 – Sorting Algorithm Selection Flowchart



§10 Bubble Sort

10.1 Definition

Bubble Sort is a simple comparison-based sorting algorithm where adjacent elements are compared and swapped if they are in the wrong order. After each full pass through the array, the largest unsorted element "bubbles up" to its correct position at the end.

10.2 Algorithm

```
Algorithm BubbleSort(A[], n):  
  for i = 0 to n-2 do           /* n-1 passes */  
    for j = 0 to n-i-2 do       /* compare pairs */  
      if A[j] > A[j+1] then  
        swap(A[j], A[j+1])     /* bubble up */  
      end if  
    end for  
  end for
```

10.3 C Implementation

```
#include <stdio.h>  
  
void bubbleSort(int A[], int n) {  
    int temp;  
    for (int i = 0; i < n-1; i++) {  
        for (int j = 0; j < n-i-1; j++) {  
            if (A[j] > A[j+1]) {  
                /* Swap A[j] and A[j+1] */  
                temp    = A[j];  
                A[j]    = A[j+1];  
                A[j+1]  = temp;  
            }  
        }  
    }  
}  
  
int main() {  
    int A[] = {64, 34, 25, 12, 22};  
    int n = 5;  
    bubbleSort(A, n);  
    printf("Sorted: ");  
    for (int i = 0; i < n; i++) printf("%d ", A[i]);  
}
```

```

return 0;
}

```

10.4 Step-by-Step Trace: {64, 34, 25, 12, 22}

Pass	Array After Pass	Element Fixed	Swaps
Pass 1	34, 25, 12, 22, 64	64 at index 4	4
Pass 2	25, 12, 22, 34 , 64	34 at index 3	3
Pass 3	12, 22, 25 , 34, 64	25 at index 2	2
Pass 4	12, 22 , 25, 34, 64	22 at index 1	0

Fig 10.1 – Bubble Sort Pass Diagram: {64, 34, 25, 12, 22}



Total Comparisons = $(n-1) + (n-2) + \dots + 1 = n(n-1)/2 = 5 \times 4 / 2 = 10$
 Each pass fixes the largest remaining element at its correct position

Bubble Sort Complexity

Best Case (sorted array) : $O(n)$ - with early termination flag

Average Case : $O(n^2)$

Worst Case (reverse sorted) : $O(n^2)$

Space Complexity : $O(1)$ - in-place

Number of Comparisons : $n(n-1)/2$

Stable Sort : Yes

10.5 Advantages & Disadvantages

Advantages	Disadvantages
Simple to implement and understand	$O(n^2)$ is too slow for large arrays
In-place ($O(1)$ space)	Not suitable for large datasets
Stable sort	Many redundant comparisons
Can detect sorted array in $O(n)$	Rarely used in practice

§11 Selection Sort

11.1 Definition

Selection Sort divides the array into a sorted part (initially empty) and an unsorted part. In each pass, it finds the *minimum element* in the unsorted part and swaps it with the first element of the unsorted part, growing the sorted section by one element per pass.

11.2 Algorithm

```
Algorithm SelectionSort(A[], n):
  for i = 0 to n-2 do
    min_idx = i                      /* assume first is minimum */
    for j = i+1 to n-1 do
      if A[j] < A[min_idx] then
        min_idx = j                 /* update minimum index */
      end if
    end for
    swap(A[i], A[min_idx])          /* place minimum at i */
  end for
```

11.3 C Implementation

```
void selectionSort(int A[], int n) {
  int min_idx, temp;
  for (int i = 0; i < n-1; i++) {
    min_idx = i;
    for (int j = i+1; j < n; j++) {
      if (A[j] < A[min_idx])
        min_idx = j;
    }
    /* Swap */
    temp      = A[min_idx];
    A[min_idx] = A[i];
    A[i]      = temp;
  }
}
```

11.4 Step-by-Step Trace: {64, 25, 12, 22, 11}

Pass	Unsorted Part	Min Found	Swap	Array After
------	---------------	-----------	------	-------------

1	64 25 12 22 11	11 at [4]	$A[0] \leftrightarrow A[4]$	11 25 12 22 64
2	25 12 22 64	12 at [2]	$A[1] \leftrightarrow A[2]$	11 12 25 22 64
3	25 22 64	22 at [3]	$A[2] \leftrightarrow A[3]$	11 12 22 25 64
4	25 64	25 at [3]	No swap	11 12 22 25 64

Fig 11.1 – Selection Sort Working Diagram: {64, 25, 12, 22, 11}

Pass 1: Find min in [0..4]



Pass 2: Find min in [1..4]



Pass 3: Find min in [2..4]



Final Sorted Array:



Total Comparisons = $(n-1) + (n-2) + \dots + 1 = n(n-1)/2 = 10$

Maximum $n-1$ swaps; always $O(n^2)$ regardless of input order

Selection Sort Complexity

Best Case : $O(n^2)$ – Always makes $n(n-1)/2$ comparisons

Average : $O(n^2)$

Worst Case : $O(n^2)$

Swaps : $O(n)$ – at most $n-1$ swaps (better than Bubble)

Space : $O(1)$ – in-place

Stable : No

§12 Insertion Sort

12.1 Definition

Insertion Sort builds the sorted array one element at a time. It picks the next element from the unsorted portion and inserts it into its correct position within the sorted portion, shifting larger elements to the right to make room—similar to sorting playing cards in hand.

12.2 Algorithm

```
Algorithm InsertionSort(A[], n):
  for i = 1 to n-1 do
    key = A[i]                /* element to insert */
    j = i - 1
    while j >= 0 and A[j] > key do
      A[j+1] = A[j]           /* shift right */
      j = j - 1
    end while
    A[j+1] = key              /* place key */
  end for
```

12.3 C Implementation

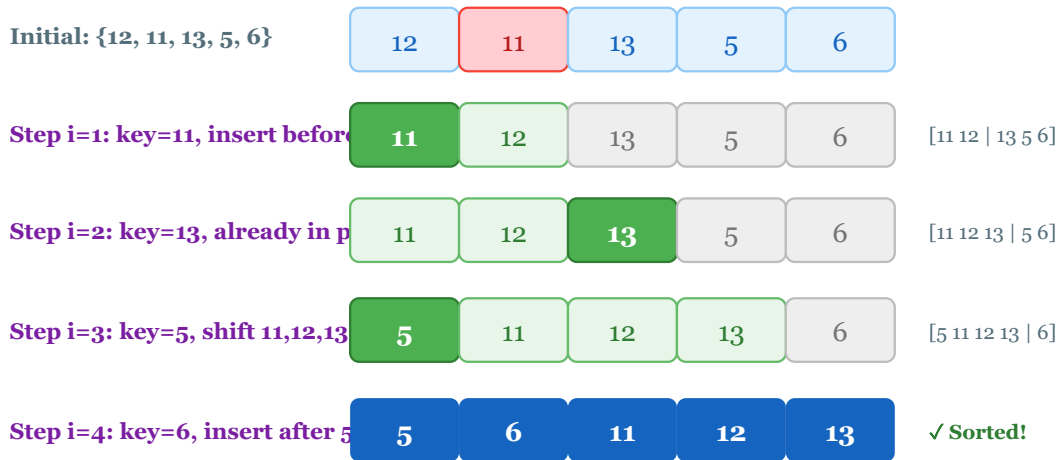
```
void insertionSort(int A[], int n) {
  int key, j;
  for (int i = 1; i < n; i++) {
    key = A[i];
    j = i - 1;
    /* Shift elements greater than key to right */
    while (j >= 0 && A[j] > key) {
      A[j+1] = A[j];
      j--;
    }
    A[j+1] = key; /* Insert key at correct position */
  }
}
```

12.4 Step-by-Step Trace: {12, 11, 13, 5, 6}

Step i	Key	Sorted Portion	After Insertion
i=1	11	[12]	11, 12, 13, 5, 6

i=2	13	[11, 12]	11, 12, 13, 5, 6
i=3	5	[11, 12, 13]	5, 11, 12, 13, 6
i=4	6	[5, 11, 12, 13]	5, 6, 11, 12, 13

Fig 12.1 – Insertion Sort Step Diagram: {12, 11, 13, 5, 6}



Working Principle: Like sorting playing cards — pick one card and insert at correct position
Best for nearly-sorted arrays; adaptive algorithm

Insertion Sort Complexity

Best Case (sorted array) : $O(n)$ - Only $n-1$ comparisons, no shifts

Average Case : $O(n^2)$

Worst Case (reverse sorted) : $O(n^2)$

Space Complexity : $O(1)$ - in-place

Stable Sort : Yes

Adaptive : Yes - faster on nearly sorted arrays

2-Mark Questions

Q 1. What is a data structure? 2M

A data structure is a way of organizing, storing, and managing data in a computer so that operations on the data can be performed efficiently. It defines both the data layout and the operations applicable to the data. Examples include arrays, linked lists, stacks, queues, trees, and graphs. The choice of data structure directly impacts algorithm performance.

Q 2. What is an Abstract Data Type (ADT)? 2M

An ADT is a mathematical model for data types that defines the data type by its behavior (operations) from the user's perspective, hiding the implementation details. It specifies *what* operations can be performed, not *how* they are implemented. Examples: Stack ADT (push, pop, peek, isEmpty), Queue ADT (enqueue, dequeue). ADTs promote modularity and abstraction in programming.

Q 3. Define Big-O notation. 2M

Big-O notation is a mathematical notation that describes the upper bound of an algorithm's time or space complexity as a function of input size n . It represents the worst-case scenario. Formally, $T(n) = O(f(n))$ if there exist constants c and n_0 such that $T(n) \leq c \cdot f(n)$ for all $n \geq n_0$. Examples: $O(1)$ —constant, $O(n)$ —linear, $O(n^2)$ —quadratic, $O(\log n)$ —logarithmic.

Q 4. What is the prerequisite for binary search? 2M

Binary search requires the array to be *sorted* in ascending or descending order before the search begins. Additionally, the data structure must support random access (like an array) so the middle element can be accessed in $O(1)$ time. Without a sorted array, binary search cannot correctly eliminate half the search space in each step, making it invalid.

Q 5. What is the difference between time complexity and space complexity? 2M

2M

Time complexity measures the number of operations an algorithm performs as a function of input size n —it tells us how fast the algorithm runs. Space complexity measures the amount of memory an algorithm uses as a function of n . Both are expressed using Big-O notation. Time complexity is usually the primary concern, but in memory-limited systems, space complexity becomes equally important. For example, Merge Sort has $O(n \log n)$ time but $O(n)$ space, while Bubble Sort has $O(n^2)$ time but $O(1)$ space.

Q 6. What is a stable sorting algorithm? 2M

A sorting algorithm is stable if it preserves the relative order of elements that have equal keys. For example, if two elements A and B have the same value, and A appears before B in the original array, in a stable sort A will still appear before B in the sorted array. Stable sorts: Bubble Sort, Insertion Sort, Merge Sort. Unstable sorts: Selection Sort, Quick Sort. Stability matters when sorting records with multiple fields.

5-Mark Questions

Q 7. Explain binary search with algorithm, example, and diagram. 5M

Binary Search is an efficient searching technique that works on sorted arrays using the divide-and-conquer approach.

Algorithm: Set $low=0$, $high=n-1$. While $low \leq high$, compute $mid=(low+high)/2$. If $A[mid]==key$, return mid (found). If $A[mid] < key$, set $low=mid+1$ (search right half). Else set $high=mid-1$ (search left half). If loop ends without finding, return -1.

Example: Array = {10, 20, 30, 40, 50}, key=30.

Step 1: $low=0$, $high=4$, $mid=2$, $A[2]=30=key \rightarrow$ Found at index 2!

Working: Each iteration halves the search space. For $n=8$: max comparisons = $\log_2(8) = 3$ only, vs 8 for linear search.

Complexity: Best $O(1)$, Average $O(\log n)$, Worst $O(\log n)$, Space $O(1)$.

Advantages: Very fast for large sorted arrays. Much better than linear search ($O(\log n)$ vs $O(n)$).

Disadvantages: Array must be sorted. Requires random access (not suitable for linked lists). Overhead of sorting makes it inefficient for single searches on unsorted data.

Q 8. Explain bubble sort with algorithm, example, and complexity analysis.

5M

Bubble Sort is a simple comparison-based sorting algorithm.

Principle: Repeatedly compare adjacent elements and swap if out of order. After each pass, the largest element "bubbles up" to its correct position at the end.

Algorithm: Outer loop i from 0 to $n-2$ (passes). Inner loop j from 0 to $n-i-2$. If $A[j] > A[j+1]$, swap them.

Example: Array = {5, 3, 8, 1, 2}

Pass 1: Compare pairs (5,3)→swap, (5,8)→no, (8,1)→swap, (8,2)→swap → {3,5,1,2,8}

Pass 2: {3,1,2,5,8}

Pass 3: {1,2,3,5,8}

Pass 4: {1,2,3,5,8} — Done.

Complexity Analysis:

- Best Case: $O(n)$ [already sorted, using flag optimization]
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$ [reverse sorted]
- Total comparisons: $n(n-1)/2$
- Space: $O(1)$ in-place. Stable: Yes.

Advantages: Simple, in-place, stable, detects sorted array early.

Disadvantages: Very slow for large arrays; many redundant comparisons.

Q 9. Compare the three sorting algorithms: Bubble, Selection, and Insertion Sort.

5M

Bubble Sort: Works by repeatedly swapping adjacent out-of-order elements. After each of $n-1$ passes, the largest element settles at the end. Makes $O(n^2)$ comparisons and $O(n^2)$ swaps in worst case. Stable, in-place. Best for: small datasets, educational purposes.

Selection Sort: Finds the minimum element from the unsorted portion and places it at the beginning. Makes $O(n^2)$ comparisons but only $O(n)$ swaps—far fewer swaps than Bubble Sort. Not stable, in-place. Best for: when write operations (swaps) are expensive, e.g., EEPROM/flash memory.

Insertion Sort: Builds the sorted array one element at a time by inserting each new element into its correct position. Most efficient of the three for nearly-sorted data ($O(n)$ best case). Stable, in-place, adaptive. Best for: small or nearly-sorted arrays; used as base case in Merge Sort and QuickSort for small subarrays.

Verdict: For nearly sorted data: Insertion Sort best. For minimizing writes: Selection Sort. In general, all three are $O(n^2)$ and inferior to Merge Sort / QuickSort for large datasets. Insertion Sort is the most practical among the three.

Q 10. Explain time complexity analysis with examples and the significance of Big-O notation. 5M

Time Complexity quantifies how an algorithm's running time grows with input size n , independent of hardware.

Why Big-O? Big-O gives the worst-case upper bound, allowing fair comparison of algorithms on any machine. It ignores constants and lower-order terms.

Examples:

- $O(1)$: Accessing $A[3]$ in an array — always 1 step.
- $O(n)$: Linear Search — in worst case visits all n elements.
- $O(\log n)$: Binary Search — halves search space each step; for $n=1000$, only ~ 10 steps.
- $O(n^2)$: Bubble Sort — nested loops, $n \times n$ comparisons.
- $O(n \log n)$: Merge Sort — the optimal comparison-based sorting bound.

Counting Operations Example:

```
for(i=0; i<n; i++) for(j=0; j<n; j++) x++;
```

→ Inner loop runs n times, outer loop runs n times → n^2 total steps → $O(n^2)$.

Significance: For $n=10^6$: $O(\log n) \approx 20$ steps; $O(n)=10^6$ steps; $O(n^2)=10^{12}$ steps (impossible in real time). Choosing the right algorithm is critical for scalable systems.

Q 11. Explain insertion sort with a detailed example and step-by-step trace. 5M

Insertion Sort works similarly to sorting playing cards: pick one card (key) and insert it into the correct position among already-sorted cards.

Algorithm: Start from $i=1$. For each i , store $A[i]$ as key. Shift all elements greater than key one position to the right. Insert key at the correct position.

Detailed Trace: A = {9, 5, 7, 1, 3}

– i=1, key=5: shift 9 right → [_, 9, 7, 1, 3] → insert 5 → [5, 9, 7, 1, 3]

– i=2, key=7: shift 9 right → [5, _, 9, 1, 3] → insert 7 → [5, 7, 9, 1, 3]

– i=3, key=1: shift 9,7,5 right → [_, 5, 7, 9, 3] → insert 1 → [1, 5, 7, 9, 3]

– i=4, key=3: shift 9,7,5 right → [1, _, 5, 7, 9] → insert 3 → [1, 3, 5, 7, 9]

Sorted: {1, 3, 5, 7, 9} ✓

Complexity: Best $O(n)$, Average $O(n^2)$, Worst $O(n^2)$. Space $O(1)$. Stable: Yes. Adaptive: Yes (faster on partially sorted arrays).

Advantage over Bubble/Selection: Best-case $O(n)$ makes it ideal when input is nearly sorted.

§14 Quick Revision Sheet

One-page summary for last-minute exam revision — Unit I: Linear Data Structures

Key Definitions

- **Data Structure:** Organized way to store & access data
- **ADT:** Interface (what) hidden from implementation (how)
- **Linear DS:** Sequential arrangement of elements
- **Big-O:** Worst-case upper bound of complexity
- **Stable Sort:** Preserves relative order of equal elements
- **In-place:** $O(1)$ extra space (no additional array)

Complexity Summary

Algorithm	Best	Worst
Linear Search	$O(1)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$
Bubble Sort	$O(n)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$

Algorithm Steps (Exam Ready)

Linear Search: for $i=0$ to $n-1$: if $A[i]=key$ return i ; return -1

Binary Search: while $low \leq high$: $mid = (l+h)/2$; if match return mid ; else narrow range

Bubble Sort: for i : for j to $n-i-1$: if $A[j] > A[j+1]$ swap

Selection Sort: for i : find min in $[i..n-1]$; swap with $A[i]$

Insertion Sort: for $i=1$: $key=A[i]$; shift right while $A[j] > key$; $A[j+1]=key$

Important Formulas

- Bubble comparisons: $n(n-1)/2$
- Binary search max steps: $\lceil \log_2(n+1) \rceil$
- Array address: **Base** + $i \times \mathbf{size}$
- Nested loops $\rightarrow O(n^2)$
- Loop where i doubles $\rightarrow O(\log n)$
- $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2)$

Sorting Comparison (Exam Table)

Property	Bubble	Selection	Insertion
----------	--------	-----------	-----------

Best Case	$O(n)$	$O(n^2)$	$O(n)$
Worst Case	$O(n^2)$	$O(n^2)$	$O(n^2)$
Space	$O(1)$	$O(1)$	$O(1)$
Stable	✓ Yes	✗ No	✓ Yes
Adaptive	✓ Yes	✗ No	✓ Yes
Swaps	$O(n^2)$	$O(n)$	$O(n^2)$

Exam Tips:

- Binary search is $O(\log n)$ because each comparison halves the search space: $2^k = n \rightarrow k = \log_2 n$ steps.
- Selection Sort always makes $O(n^2)$ comparisons regardless of input—no best-case optimization.
- Insertion Sort is best for nearly-sorted or small arrays. Real compilers use it for small subarrays in Timsort.
- All three basic sorts use $O(1)$ space—they are in-place algorithms.
- ADT separates interface from implementation—a key concept in Object-Oriented Programming.

Primary Textbooks

1. **Mark Allen Weiss** – *Data Structures and Algorithm Analysis in C*, Pearson Education, 2nd Edition.
Covers ADTs, sorting, searching, and complexity analysis with rigorous mathematical treatment.
Highly recommended for B.Tech students.
2. **Ellis Horowitz & Sartaj Sahni** – *Fundamentals of Data Structures in C*, Silicon Press, 2008.
Classic textbook with comprehensive coverage of all linear and non-linear data structures with C implementations.

Reference Books

1. **Kurt Mehlhorn & Peter Sanders** – *Algorithms and Data Structures: The Basic Toolbox*, Springer.
Excellent for understanding the theoretical foundations of algorithm design and analysis.
2. **Aho, Ullman & Hopcroft** – *C Data Structures and Algorithms*, Addison-Wesley.
Foundational text with formal proofs and comprehensive algorithm analysis.
3. **Brad Miller & David Ranum** – *Problem Solving with Algorithms and Data Structures*, Franklin, Beedle & Associates.
Practical, beginner-friendly approach with visual examples and interactive exercises.
4. **Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest & Clifford Stein** – *Introduction to Algorithms (CLRS)*, MIT Press, 3rd Edition.
The definitive reference for algorithm analysis and design; used in top universities worldwide.
5. **Robert Sedgewick** – *Algorithms in C*, Addison-Wesley.
Practical algorithms with C implementations; excellent coverage of sorting and searching.

Online Resources:

- GeeksforGeeks – Algorithm Visualizations: www.geeksforgeeks.org
- NPTEL – Data Structures Course (IIT lectures)
- Visualgo – Algorithm Visualizer: visualgo.net
- MIT OpenCourseWare – 6.006 Introduction to Algorithms

Data Structures – Unit I: Linear Data Structures

B.Tech 1st Year | Computer Science & Engineering

Prepared for complete coverage of Unit I Syllabus

14 Sections

10 Diagrams

11 Q&A

Quick Revision